Aquí tienes una **Base de Conocimiento e Inteligencia del Proyecto** diseñada para servir como memoria técnica centralizada (tanto para los miembros de tu equipo como para los agentes de programación que utilices) junto con la **hoja de ruta de desarrollo, arquitectura de código auditable y estructura del informe IEEE**.

PARTE 1: Base de Conocimiento y Estado del Arte (Memoria para Agentes)

1. Naturaleza y Complejidad del Problema (*Nurse Rostering Problem* - NRP)

- **Importancia crítica del dominio:** La asignación de personal sanitario (*Nurse Scheduling* o *Nurse Rostering*) gestiona el recurso más delicado y costoso de un hospital, donde los errores de planificación o la falta de cobertura afectan directamente a la seguridad del paciente, incrementan los errores médicos y provocan agotamiento (*burnout*) en la plantilla.
- **Diferencias frente a otros sectores:** A diferencia del comercio o la industria, un hospital opera de forma ininterrumpida (24/7) sin posibilidad de posponer urgencias, bajo estrictas normativas laborales de descanso y requisitos obligatorios de cualificación (*skill-mix*) por turno.
- **Complejidad computacional:** El NRP está clasificado como un problema de optimización combinatoria **NP-hard**. La combinación de múltiples tipos de turnos, horizontes de planificación largos (ej. 28 días), restricciones solapadas y criterios objetivos en conflicto provoca que el espacio de búsqueda crezca exponencialmente con el número de enfermeros, haciendo inviable su resolución manual.

2. Análisis del Estado del Arte (SoA) en Asignación de Turnos

Para fundamentar la sección de revisión de literatura del informe (recomendada en las directrices del proyecto) y guiar el diseño algorítmico de otros agentes, el estado del arte del NRP se estructura en tres grandes paradigmas:

A. Formulaciones Exactas (Programación Lineal Entera Mixta - MILP)

1. **Modelado de Asignación Indexada vs. Basado en Patrones:**

- *Asignación directa (Indexada por Día-Turno):* Define variables binarias $x_{n,d,s} \in \{0,1\}$ que indican si el enfermero n trabaja el día d en el turno s. Es la formulación más flexible para incorporar restricciones de ventana deslizante y peticiones individuales en motores como SCIP, CBC o HiGHS dentro de Google OR-Tools.
- *Generación de Columnas (Branch-and-Price):* En la literatura académica avanzada, se pregeneran patrones completos para cada enfermero y se resuelve un problema maestro de particionamiento de conjuntos (*Set Partitioning*). Sin embargo, dado el límite estricto de tiempo de ejecución en torneo (120 segundos incluyendo la construcción del modelo dentro del método run()), una formulación MILP compacta y bien ajustada mediante preprocesado de índices suele ser más robusta y directa de defender.

2. **Linealización de Penalizaciones (Restricciones Blandas / *Goal Programming*):**
   - En el estado del arte (como las formulaciones derivadas de las competiciones internacionales *INRC-I* e *INRC-II*), las desviaciones respecto al horario ideal se modelan mediante **variables de holgura (*slack/surplus variables*) no negativas**.
   - Si una cantidad Q debe situarse en el rango $\[L, U\]$, se introducen dos variables continuas (o enteras) no negativas $u \geq 0$ (defecto) y $o \geq 0$ (exceso) tales que:

     $$Q + u \geq L \text{ y } Q - o \leq U$$

     sumando $(u + o)$ directamente en la función objetivo a minimizar.
3. **Modelado de Consecutividad (Rachas de Trabajo y Descanso):**
   - **Máximo de días consecutivos (M_max):** Se modela de forma exacta y muy eficiente con una ventana deslizante de longitud $M_max + 1$. Por cada ventana que exceda el límite, una variable de exceso captura la violación.
   - **Mínimo de días consecutivos (M_min):** En la literatura de NRP se modela mediante restricciones por bloques de longitud $k \in \{1, \ldots, M_min - 1\}$ acotados por días libres antes y después (transición de descanso a trabajo y de trabajo a descanso),

penalizando con peso $(M_m in - k)$ cuando aparece una racha aislada de tamaño k.

4. **Reducción de Simetrías y Ajuste del Solver:**
    - No crear variables x_n, d, s cuando el enfermero n carece de la habilidad requerida por el turno s reduce drásticamente el tamaño de la matriz de restricciones y acelera el árbol de *Branch-and-Cut*.

B. Metaheurísticas y Algoritmos Evolutivos / Meméticos

1. **Representación de Soluciones y Manejo de Factibilidad:**
    - El estándar en el torneo exige devolver una lista de listas de cadenas de texto (N enfermeros $\times$ D días, usando "OFF" para días libres).
    - A nivel interno, los agentes pueden representar el cromosoma como una matriz entera de NumPy $N \times D$ (donde $0 = OFF$ y $1 \dots S$ son los IDs de turno) para evaluar la función de *fitness* de forma vectorizada y ultrarrápida, convirtiéndola al formato de lista de strings únicamente en get_best_solution().
2. **Estrategia de Reparación (*Repair Operator*) vs. Penalización:**
    - En el NRP, los operadores clásicos de cruce y mutación suelen romper las restricciones duras o empeorar la cobertura diaria.
    - El propio *baseline* del torneo utiliza un operador de fijación/reparación tras el cruce y la mutación para garantizar la factibilidad y acercar la asignación a la demanda preferida de enfermeros por turno y día. Diseñar un **operador de reparación inteligente (heurístico/greedy)** que reasigne turnos considerando las habilidades (skills), días libres pedidos y rachas de cada trabajador es el factor diferencial para alcanzar el cuartil Q1.
3. **Operadores Genéticos Específicos del Dominio:**
    - *Cruces orientados a filas (Enfermeros):* Intercambiar horarios completos de subconjuntos de enfermeros entre dos padres mantiene intactas las secuencias consecutivas individuales (fines de semana, rachas de días libres y patrones no deseados), requiriendo solo reparar la cobertura vertical por día.

- *Cruces orientados a columnas (Ventanas de Días / Fines de Semana):* Intercambiar bloques semanales completos mantiene intacta la factibilidad diaria y la cobertura de demanda vertical, requiriendo solo evaluar la transición en la frontera entre semanas.
- *Hibridación Memética (GA + Búsqueda Local / Simulated Annealing):* Aplicar una fase corta de búsqueda local (*Hill Climbing* o *Simulated Annealing* con movimientos de tipo *Swap* entre dos enfermeros en un mismo día o *Shift Change*) sobre los mejores individuos permite pulir violaciones locales de patrones prohibidos sin alterar la cobertura global.

C. Líneas Bibliográficas Clave para Citar en la Sección *Introduction / References*

Para cumplir con la revisión de artículos académicos valorada en el proyecto, el equipo o el agente encargado de redactar el estado del arte debe referenciar:

- **Revisiones Taxonómicas Fundamentales:** Burke et al. (2004) *"The state of the art of nurse rostering"* (*Journal of Scheduling*); Ernst et al. (2004) *"Staff scheduling and rostering: A review of applications, methods and models"* (*European Journal of Operational Research*); Van den Bergh et al. (2013) *"Personnel scheduling: A literature review"*.
- **Competiciones Internacionales (INRC):** Haspeslagh et al. (2014) *"The first international nurse rostering competition 2010"* y Ceschia et al. (2019) *"The second international nurse rostering competition"*, cuyas estructuras de restricciones blandas (contratos, secuencias consecutivas, fines de semana completos y patrones prohibidos) son idénticas a las del proyecto.

3. Especificación Técnica del Problema del Proyecto (Reglas de Negocio)

A. Estructura de Datos de Entrada (JSON de Instancias)

Se proporcionan 7 instancias de prueba para experimentación local. Cada instancia contiene:

- **Horizonte temporal:** Número de días a planificar (ej. "number of days": 28, equivalente a 4 semanas de lunes a domingo).

- **Catálogo de habilidades (skills):** Conjunto de cualificaciones del hospital, como ["Nurse", "HeadNurse"].
- **Turnos diarios (shifts):** Identificador del turno (id / shift_id, ej. "E", "L", "D", "DH"), hora de inicio y fin (start_time, end_time), habilidad requerida (needed_skill / skills) y vector de demanda estimada o número preferido de enfermeros por cada día del horizonte (demand / preferred_number_nurses).
- **Plantilla de enfermeros (nurses):** Cada trabajador tiene asociado su nurse_id (id), su conjunto de habilidades (skills), y los parámetros de su contrato y preferencias:
    - min_assignments y max_assignments: Rango ideal de días totales trabajados en el periodo.
    - min_consecutive_days y max_consecutive_days: Rango ideal de días consecutivos trabajando por racha.
    - min_consecutive_free_days y max_consecutive_free_days: Rango ideal de días libres consecutivos por descanso.
    - complete_weekends: Booleano que indica si, en caso de trabajar en fin de semana, debe trabajar ambos días (sábado y domingo).
    - same_shift_weekend: Booleano que indica si, en caso de trabajar un fin de semana, debe realizar el mismo tipo de turno en ambos días.
    - unwanted_patterns: Lista de secuencias de turnos consecutivos a evitar (ej. ["L-D", "D-E-D"] o "N-N-N", "N-E").
    - days_off_requests (days_off): Lista de índices de días en los que el enfermero ha solicitado librar.
    - shift_off_requests (shift_off): Lista de pares {"day": d, "shift_type": s} en los que el enfermero solicita evitar un turno específico en un día concreto.

B. Restricciones Duras (*Hard Constraints* - Factibilidad Obligatoria)

Un horario es **factible** única y exclusivamente si cumple estas dos condiciones en todo momento:

1. **Cobertura mínima cualificada:** Para cada día y cada tipo de turno, debe haber asignado **al menos un miembro del personal** que posea la habilidad (skill) requerida por dicho turno.
2. **Unicidad de turno diario:** Un miembro del personal **no puede ser asignado a más de un turno en el mismo día**.

C. Restricciones Blandas y Función Objetivo (*Soft Constraints*)

El objetivo es **minimizar la desviación total respecto al horario ideal**, sumando la desviación global de demanda más las desviaciones individuales de todos los trabajadores:

1. **Desviación de demanda por turno y día:** Diferencia en valor absoluto entre el número de enfermeros asignados al turno s el día d y la demanda estimada (tanto asignar de menos como asignar de más penaliza por cada trabajador de desviación).
2. **Desviación en el número total de asignaciones:** Días que faltan para llegar a min_assignments o días que exceden max_assignments en el horizonte completo para cada enfermero.
3. **Desviación en días de trabajo consecutivos:** Por cada racha de trabajo continua de un enfermero, días que faltan para alcanzar min_consecutive_days o días que superan max_consecutive_days.
4. **Desviación en días libres consecutivos:** Por cada periodo de descanso de un enfermero, días libres que faltan para llegar a min_consecutive_free_days o que exceden max_consecutive_free_days.
5. **Fines de semana completos (complete_weekends):** Si está activo para el enfermero, se suma 1 día de desviación por cada fin de semana en que trabaje solo el sábado o solo el domingo.
6. **Mismo turno en fin de semana (same_shift_weekend):** Si está activo y el enfermero trabaja en el fin de semana, se penaliza la desviación si realiza turnos distintos entre el sábado y el domingo.
7. **Patrones no deseados (unwanted_patterns):** Se suma 1 unidad de desviación por cada aparición de una secuencia prohibida en el calendario del enfermero.

8. **Peticiones de día libre (days_off_requests):** Se suma 1 día de desviación por cada día solicitado como libre en el que se le asigne cualquier turno al enfermero.

9. **Peticiones de turno libre (shift_off_requests):** Se suma 1 día de desviación por cada ocasión en que se asigne al enfermero un turno específico que había solicitado evitar en ese día concreto.

PARTE 2: Estructura de Desarrollo y Estándar de Código "Auditable"

La evaluación final multiplica **Agencia Humana × Calidad del Trabajo** (Human agency × Work quality). En la entrevista técnica (10 minutos de presentación ejecutiva + 20-30 minutos de interrogatorio técnico individual), cualquier miembro del equipo debe poder explicar cada restricción matemática, cada línea de Python y el porqué de cada decisión de diseño o hiperparámetro.

1. Arquitectura de Archivos a Desarrollar

A. Módulo 1: milp.py (Clase MILP)

- **Reglas estrictas de interfaz:** No se pueden modificar las cabeceras de __init__, read_problem_instance, get_best_solution ni run, o el algoritmo quedará descalificado del torneo.

- **Flujo interno obligatorio:**

    - __init__(self, time_deadline, problem_path, solver_type='SCIP', **kwargs): Inicializa atributos y configura parámetros opcionales cuyos valores por defecto deben ser los mejores hallados experimentalmente.

    - read_problem_instance(self, problem_path): Lee el archivo JSON y almacena los datos en estructuras internas. **Prohibido** precomputar, buscar o construir el modelo aquí.

    - run(self): Mantiene la llamada obligatoria a self.read_problem_instance(self.problem_path) en la primera línea. Instancia el solver con pywraplp.Solver.CreateSolver(self.solver_type) (usando la sintaxis de Constraint y SetCoefficient trabajada en los laboratorios), mide el tiempo de construcción t2 - t1, descuenta ese tiempo del límite (remaining_time = int(self.time_deadline - (t2 - t1))),

aplica solver.SetTimeLimit(remaining_time * 1000) en milisegundos, ejecuta solver.Solve() y guarda la solución codificada en self.best_solution.

- get_best_solution(self): Devuelve self.best_solution como lista de listas de strings ("OFF" para días sin turno asignado).

- **Descomposición modular recomendada:** Divide la construcción del modelo dentro de la clase en métodos privados claramente nombrados (_create_decision_variables(), _add_hard_constraints(), _add_demand_deviation_constraints(), _add_workload_constraints(), _add_consecutive_days_constraints(), _add_weekend_and_pattern_constraints()).

B. Módulo 2: metaheuristic.py (Clase Metaheuristic)

- **Reglas estrictas de interfaz:** Mismas restricciones de cabeceras intactas (__init__, read_problem_instance, get_best_solution, run). Los hiperparámetros óptimos obtenidos en los experimentos deben quedar fijados como valores por defecto en el constructor __init__.

- **Superar el *Baseline* del Torneo:** El algoritmo debe mejorar la Búsqueda Aleatoria y el Algoritmo Genético base del torneo (población de 30 individuos, torneo binario de tamaño 10, cruce de un punto al 70%, mutación del 20% alterando el 5% de los genes, operador de reparación de demanda preferida y elitismo).

- **Descomposición modular recomendada:** Aísla cada fase evolutiva en métodos independientes:

  - _initialize_population(): Generación constructiva heurística factible.

  - _evaluate_fitness() y _delta_evaluation(): Cálculo desglosado por cada una de las 9 componentes de la función objetivo.

  - _select_parents(): Torneo k-ario o selección por rango.

  - _crossover(): Cruce por bloques de días o intercambio de filas de enfermeros.

  - _mutate(): Mutaciones dirigidas (eliminar patrones prohibidos, conceder días libres solicitados, completar fines de semana).

- _repair_schedule(): Operador que garantiza en todo momento las dos restricciones duras (1 enfermero con *skill* por turno/día y máximo 1 turno por día/enfermero).

C. Módulo 3: experiments.py (Suite Experimental Replicable)

- **Requisitos de ejecución:** 120 segundos por ejecución de cada modelo/metaheurística; métrica principal basada en el mejor valor de la función objetivo (y tiempo de cómputo en caso de empate).
- **Rigor estadístico:** Cada configuración debe evaluarse *n* veces (fijando semillas aleatorias) y **utilizando exactamente las mismas poblaciones iniciales entre distintas configuraciones** de la metaheurística para garantizar comparaciones justas.
- **Optimización de hiperparámetros:** Debe incluir el código ejecutable de **Grid Search, Random Search o Bayesian Optimization** para la metaheurística, y la comparativa de distintos modelos MILP, *solvers* (SCIP, CBC, HiGHS) y parámetros de MPSolverParameters (PRESOLVE, LP_ALGORITHM, SCALING, RELATIVE_MIP_GAP).

2. Estándar de Documentación de Código para la Defensa ("Conciencia de Parámetros")

Para que cualquier miembro del equipo pueda responder al instante durante la entrevista técnica sin perder tiempo buscando archivos, establece un bloque de configuración centralizado al inicio de milp.py y metaheuristic.py, documentando cada parámetro ajustable con esta plantilla en los *docstrings*:

Python

```
#
================================================================================
==
# FICHA DE CONCIENCIA DE PARÁMETROS AJUSTABLES (DEFENSA TÉCNICA)
#
================================================================================
==
# Parámetro: pop_size (int) = 50 [Valor por defecto optimizado en Exp. 3.2]
# - ¿Qué controla?: Número de horarios factibles mantenidos en cada generación.
# - Rango válido: [10, 200]. (Baseline del torneo = 30).
# - Impacto si se SUBE manualmente (ej. >100): Mayor diversidad genética (exploración),
#   pero reduce drásticamente el número de generaciones ejecutadas en los 120s.
```

```
# - Impacto si se BAJA manualmente (ej. <20): Más generaciones por minuto (explotación
#   rápida), pero alto riesgo de convergencia prematura en óptimos locales.
# - Vínculo matemático/Informe: Sección V-B, Tabla III del informe IEEE.
#
================================================================================
==
```

Además, cada método de restricción en milp.py debe incluir en su comentario la ecuación matemática exacta en notación compacta que aparece en el informe IEEE, permettant una trazabilidad 1:1 entre código y documento.

PARTE 3: Estructura Detallada del Informe IEEE (Máximo 10 Páginas)

Para obtener el 90-100% en calidad de documento y diseño, el informe debe respetar estrictamente la plantilla IEEE de doble columna, no superar las 10 páginas e incluir todas las secciones obligatorias con figuras y tablas coherentes:

1. Abstract (~0.25 páginas)

- Resumen ejecutivo autocontenido: contexto del *Nurse Rostering Problem*, formulación de los modelos MILP propuestos, diseño de la metaheurística evolutiva/híbrida, metodología experimental sobre las 7 instancias (límite de 120 s) y principales mejoras porcentuales de *fitness* y *ranking* frente a los *baselines*.

2. Introduction (~1.25 páginas)

- **Motivación y contexto:** Importancia estratégica de la planificación de enfermería en hospitales (operación 24/7, impacto en seguridad clínica, leyes laborales y *skill-mix*) y su complejidad NP-hard.
- **Estado del Arte (Literature Review):** Síntesis crítica de artículos académicos sobre métodos exactos (MILP, relajaciones) y metaheurísticas (GA, algoritmos meméticos, operadores de reparación) aplicados al NRP.
- **Objetivos y estructura del artículo:** Presentación de las contribuciones del equipo y organización del documento.

3. Use of Generative AI (~0.75 páginas)

- **Transparencia y Auditoría Crítica (Clave para la nota de *Human Agency*):** Detallar con total honestidad en qué fases se empleó IA

generativa (ej. búsqueda bibliográfica, estructuración de código base, generación de gráficas).

- **Registro de Fallos de la IA y Corrección Humana:** Documentar casos concretos donde la IA falló (ej. intentos de usar sintaxis no permitida, errores de indexación en los bordes de ventanas deslizantes de días consecutivos, o generación de modelos MILP demasiado pesados que agotaban el tiempo de construcción antes de llamar a `Solve()`) y explicar cómo el equipo auditó, detectó y corrigió matemáticamente dichos errores.

4. Design of MILP Model (~2.25 páginas)

- **Notación Matemática Compacta:** Tabla formal con Conjuntos (Enfermeros N, Días D, Turnos S, Habilidades K), Parámetros de entrada y Variables de Decisión (binarias de asignación $x_{n,d,s}$, binarias auxiliares de trabajo diario $w_{n,d}$ y variables continuas/enteras de desviación).

- **Función Objetivo y Restricciones:** Formulación explícita de la función objetivo (suma de desviaciones globales e individuales), las 2 restricciones duras y la linealización detallada de las restricciones blandas.

- **Variantes del Modelo y Justificación Arquitectónica:** Explicación de las alternativas de modelado probadas (ej. Modelo Base vs. Modelo Completo vs. Modelo con poda de variables por *skills* o relajación de restricciones costosas) y qué estructuras de datos (diccionarios indexados por tuplas) se usaron para minimizar el tiempo de construcción en Python.

5. Description of Proposed Metaheuristic (~2.0 páginas)

- **Arquitectura Algorítmica:** Diagrama de flujo o pseudocódigo del bucle evolutivo completo.

- **Representación y Población Inicial:** Justificación de la estructura de datos interna frente al formato externo de torneo y método de inicialización constructiva factible.

- **Operadores de Selección, Cruce, Mutación y Reparación:** Explicación detallada y justificación de por qué se eligieron estos operadores específicos para el problema de turnos y cómo funciona el mecanismo de reparación (*fix operator*) para mantener las restricciones duras y optimizar la cobertura de demanda.

6. Experiments (~2.5 páginas)

- **Configuración Experimental y Metodología:** Descripción de las 7 instancias de prueba, límite de 120 s por ejecución, número de repeticiones *n* por configuración, uso de poblaciones idénticas entre configuraciones para control de varianza y métrica de evaluación (*Average Fitness*, *Computation Time* y *Average Rank* al estilo del torneo).

- **Experimentos MILP:** Comparativa de formulaciones, *solvers* (SCIP, CBC, HiGHS) y configuraciones de parámetros del solver frente al *Simple MILP baseline*.

- **Optimización de Hiperparámetros de la Metaheurística:** Resultados del proceso de *Grid Search*, *Random Search* o *Bayesian Optimization* (tablas de sensibilidad y gráficas de convergencia) y comparativa final frente a los *baselines* de *Random Search* y *Genetic Algorithm*.

7. Conclusions (~0.5 páginas)

- Síntesis de hallazgos empíricos, comparativa crítica entre el enfoque exacto (MILP) y el metaheurístico según el tamaño/complejidad de cada instancia, limitaciones encontradas y líneas de trabajo futuro.

8. References (~0.5 páginas)

- Bibliografía en formato IEEE estricto incluyendo los artículos del estado del arte revisados y la documentación técnica de las herramientas empleadas.

Próximos pasos sugeridos para trabajar por fases:

1. **Fase 1 (Inspección de Instancias y Validador):** Si compartes una de las 7 instancias JSON o el resto del código plantilla (milp.py / metaheuristic.py), podemos construir primero el **evaluador exacto de la función objetivo** en Python para verificar que el cálculo de todas las desviaciones coincide al 100% con los ejemplos de las diapositivas.

2. **Fase 2 (Formulación Matemática y milp.py):** Redactar las ecuaciones exactas en LaTeX para la sección *Design of MILP model* y programar la clase MILP completamente documentada.

3. **Fase 3 (Diseño de metaheuristic.py y experiments.py):** Implementar la metaheurística, el *pipeline* de optimización de hiperparámetros y las tablas para el informe IEEE.